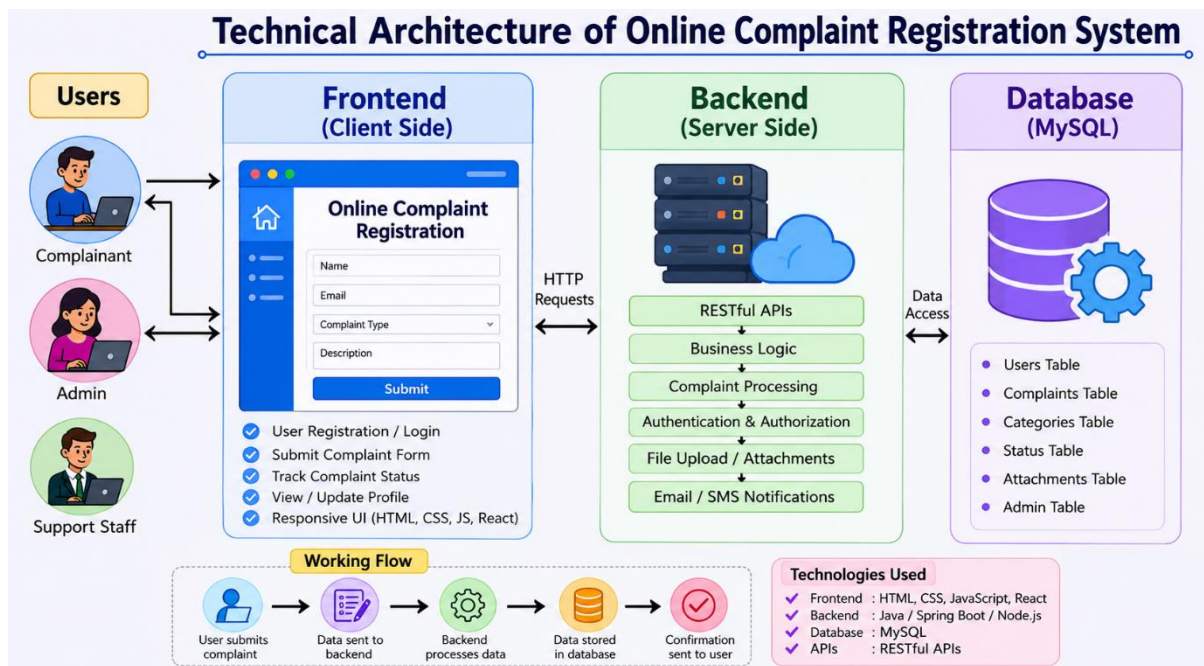


ONLINE COMPLAINT REGISTRATION AND MANAGEMENT SYSTEM

Project Documentation

Technology Stack: MongoDB · Express.js · React.js · Node.js (MERN)

TECHNICAL ARCHITECTURE



The Technical Architecture of our **Online Complaint Registration System** follows a **client-server architecture**, where the frontend works as the client and the backend works as the server.

The **frontend** provides a simple and user-friendly interface for users and admins to register complaints, view details, and track complaint status. It is developed using **HTML, CSS, JavaScript, and Bootstrap** and communicates with the backend using **REST APIs**.

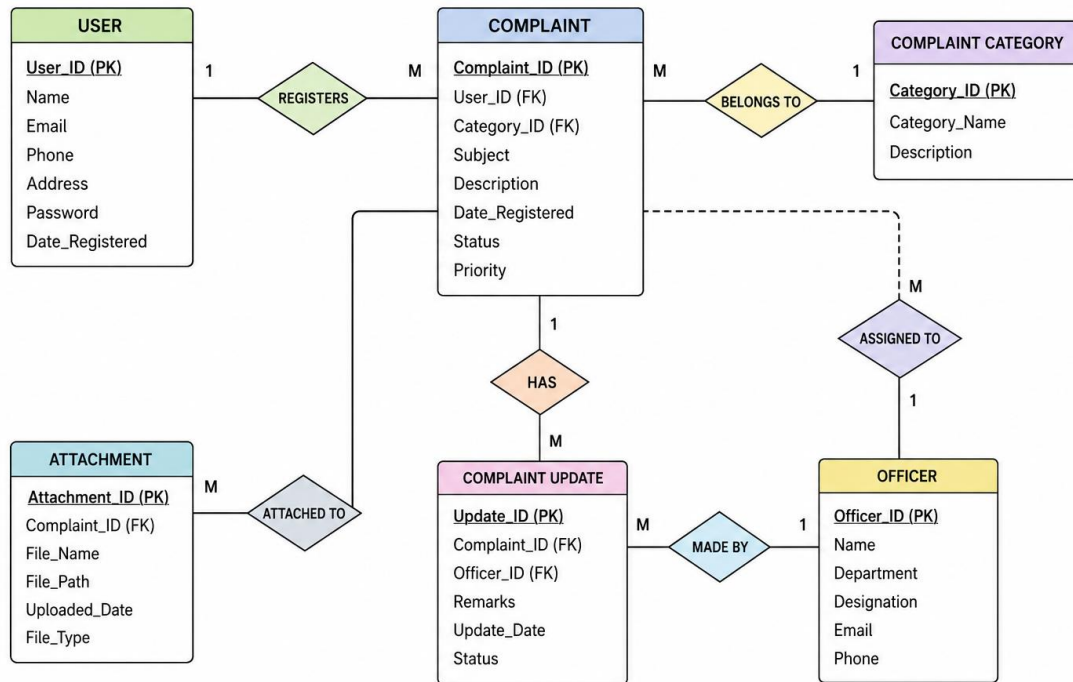
The **backend** handles the application logic, user authentication, complaint management, and data processing. It is developed using **Java with Spring Boot**.

The system uses **MySQL database** to store user information, complaint details, and status updates securely.

Together, the frontend, backend, APIs, and database provide a reliable platform for users to submit complaints and for administrators to manage and resolve them efficiently.

ER DIAGRAM

ER DIAGRAM FOR ONLINE COMPLAINT REGISTRATION SYSTEM



This ER diagram represents the Online Complaint Registration System. It shows the relationship between users, complaints, agents, and messages. Users can register and submit complaints by filling in the required details. Each complaint is assigned to an agent for review and resolution.

The diagram also includes a chat feature that allows users and agents to communicate regarding a complaint. Messages are linked using the User ID and Complaint ID to ensure proper tracking of conversations. Overall, the ER diagram helps in understanding the database structure and the flow of information within the system.

FEATURES

1. User Registration

User Registration: Users can create an account by providing their email address and password to access the complaint management system.

Profile Management: Users can maintain their personal information, such as name, contact details, and address, for easy complaint tracking.

2. Complaint Categories

Category Selection: Users can choose the appropriate complaint category before submitting their complaint.

Department Filtering: The system helps users identify the relevant department responsible for handling their issue.

3. Complaint Submission

Online Complaint Submission: Users can submit complaints by providing details of the issue and uploading supporting documents or images.

Complaint Tracking: Users can monitor the status of their complaints from submission to resolution.

4. Officer Dashboard

Complaint Assignment: Officers can view and manage complaints assigned to their department.

Status Updates: Officers can update complaint progress, add remarks, and provide resolution details.

5. Admin Management

Officer Approval: Administrators verify and approve officer accounts to ensure authorized access.

System Monitoring: Admins oversee complaint activities, manage users, and generate reports for effective system management.

6. Notifications & Communication

Automated Notifications: Users receive updates regarding complaint submission, status changes, and resolution.

User–Officer Communication: The system enables communication between users and officers for clarifications and complaint follow-up.

ROLES AND RESPONSIBILITIES

1. USER REGISTRATION

Example:

Priya wants to report a streetlight that is not working in her area. She visits the Online Complaint Registration System and creates an account by entering her name, email address, phone number, and password. After successful registration, she logs into the system.

2. SELECTING DEPARTMENT & COMPLAINT CATEGORY

Example:

Priya navigates through the available departments such as Electricity, Roads, Water Supply, and Sanitation. Since her issue relates to a faulty streetlight, she selects the Electricity Department and chooses the Streetlight Maintenance category.

3. SUBMITTING A COMPLAINT

Example:

Priya fills out the complaint form by describing the problem, providing the location details, and uploading a photo of the damaged streetlight. She then submits the complaint through the portal.

4. COMPLAINT ACKNOWLEDGEMENT

Example:

After submission, the system generates a unique complaint ID and sends a confirmation notification to Priya via email. The complaint is forwarded to the concerned department for review.

5. COMPLAINT STATUS TRACKING

Example:

Using her dashboard, Priya can monitor the complaint status, view updates from officers, and check whether the complaint is pending, under process, or resolved.

6. ADMIN VERIFICATION

Example:

The system administrator verifies officer accounts and department access rights. This process ensures that only authorized personnel can handle complaints and update records.

7. SYSTEM MONITORING

Example:

The administrator monitors complaint activities, generates reports, manages escalated cases, and ensures the smooth operation and security of the platform.

8. OFFICER COMPLAINT HANDLING

Example:

An officer from the Electricity Department reviews Priya's complaint, assigns it to a maintenance team, and updates the complaint status with remarks regarding the repair process.

9. ISSUE RESOLUTION

Example:

The maintenance team repairs the streetlight and submits proof of completion. The officer marks the complaint as resolved, and Priya receives a notification confirming that the issue has been successfully addressed.

USER FLOW

User (Citizen)

Role: Register complaints, track complaint progress, communicate with officers, and manage account details.

Flow:

Account Creation and Login

- Register using email, phone number, and password.
- Log in securely to access the complaint portal.

Complaint Registration

- Select the appropriate department and complaint category.
- Enter complaint details and upload supporting documents or images.
- Submit the complaint and receive a unique complaint ID.

Complaint Monitoring

- View all submitted complaints from the dashboard.
- Track complaint status such as Submitted, Under Review, In Progress, and Resolved.

Communication

- Exchange messages with the assigned officer regarding complaint updates.
- Provide additional information when requested.

Profile Maintenance

- Update personal information and contact details.
- View complaint history and previous resolutions.

Officer

Role: Handle assigned complaints, communicate with users, and update complaint progress.

Flow:

Registration and Login

- Create an officer account and log in using authorized credentials.

Complaint Handling

- Access assigned complaints through the officer dashboard.
- Review complaint details and supporting documents.

Complaint Processing

- Investigate issues and coordinate necessary actions.
- Add remarks, upload reports, and update complaint status.

User Communication

- Respond to user queries and provide progress updates.
 - Request additional information when needed.
-

Administrator

Role: Manage the entire complaint management system, supervise officers, and ensure efficient complaint resolution.

Flow:

System Administration

- Monitor platform activities and complaint statistics.
- Ensure smooth functioning of the system.

Complaint Allocation

- Assign complaints to appropriate officers based on department and workload.
- Track pending and escalated complaints.

User and Officer Management

- Approve officer registrations and manage user accounts.
- Control access permissions and system roles.

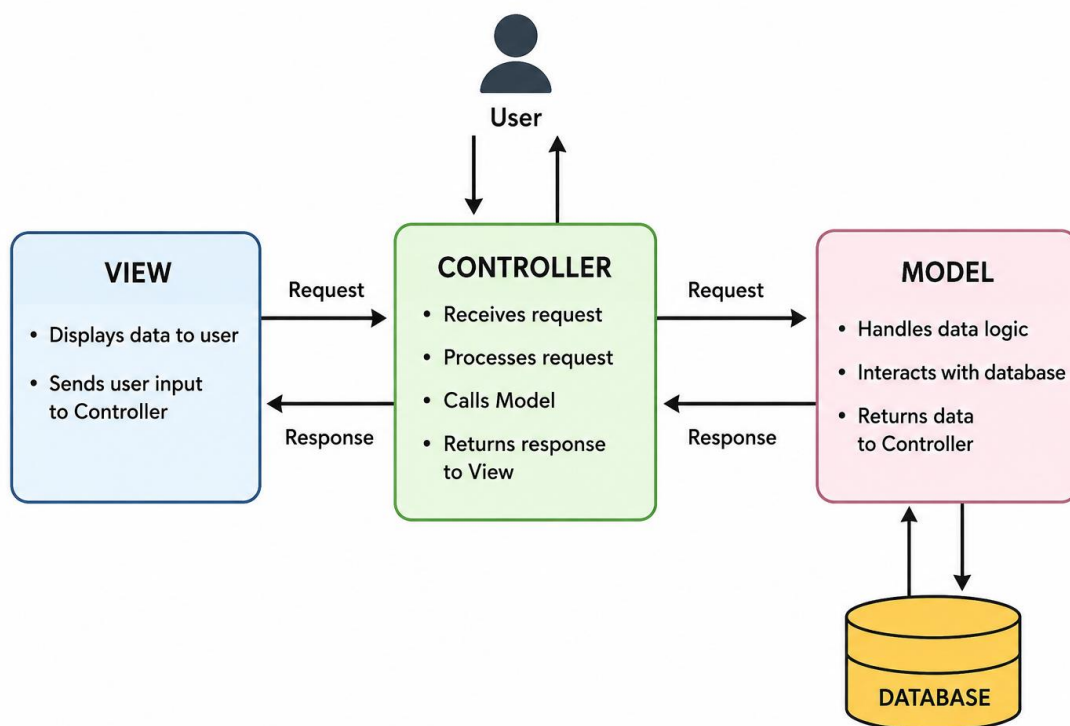
Performance Monitoring

- Review complaint resolution rates and officer performance.
- Generate reports and analytics for decision-making.

Platform Maintenance

- Enforce security policies and data protection measures.
- Implement system improvements and resolve operational issues.

MVC Pattern



MVC Architecture

The Online Complaint Registration System is developed using the **Model-View-Controller (MVC)** architecture. MVC divides the application into three independent components, making the system organized, maintainable, and easy to expand.

Model Layer (Data Management)

The **Model** layer manages all application data and database operations. It defines the structure of entities such as Users, Complaints, Officers, Messages, and Administrators. The model is responsible for storing,

retrieving, updating, and deleting information from the database while ensuring data consistency and integrity.

Controller Layer (Business Logic)

The **Controller** layer processes requests received from users and system administrators. It acts as a bridge between the View and the Model by handling business logic, validating user input, processing complaint submissions, managing complaint assignments, and updating complaint statuses. After processing the request, it returns the appropriate response to the user.

View Layer (User Interface)

The **View** layer represents the user-facing part of the application. It includes pages such as Registration, Login, Complaint Submission, Complaint Tracking, Officer Dashboard, and Admin Dashboard. The view collects user input and displays information received from the controller in a user-friendly format.

Advantages of MVC Architecture in This Project

Modular Design: Each component performs a specific function, making the application easier to understand and maintain.

Easy Maintenance: Changes in one layer have minimal impact on the other layers, reducing development complexity.

Scalability: Additional modules and features can be integrated without affecting the existing structure.

Code Reusability: Controllers and models can be reused across different modules of the system.

Improved Testing: Individual layers can be tested independently, improving software quality and reliability.

Team Collaboration: Developers can work on the user interface, business logic, and database components simultaneously, increasing productivity.

Client setup (installing react app)

Description

Setting Up the React Frontend

1. Open the **Client** folder in Visual Studio Code and launch the terminal.
2. Create a new React project using Vite by running the following command:

```
npm create vite@latest . -- --template react
```

3. When prompted, choose the following options:

- **Framework:** React
- **Variant:** JavaScript

4. Move to the client project directory:

```
cd client
```

5. Install the required dependencies:

```
npm install
```

6. Start the development server:

```
npm start
```

7. After the server starts successfully, open the URL displayed in the terminal (usually `http://localhost:5173`) to view the React application in your browser.

Server setup (npm init)

Backend Server Configuration

1. Open the **Server** folder in Visual Studio Code and launch the terminal.
2. Initialize a new Node.js project by executing the following command:

```
npm init -y
```
3. This command generates a **package.json** file with default project settings.
4. Create the main server file:

server.js

5. Organize the project structure by creating the following directories:

controllers/

models/

routes/

6. The purpose of each folder is:

- **models** – Contains database schemas and data-related logic.
- **controllers** – Handles application logic and request processing.
- **routes** – Defines API endpoints and connects them to controllers.

7. After creating the files and folders, the basic backend structure is ready for developing REST APIs and database integration.

Development and explanation

Project Setup

The development process begins by creating the server project and initializing it using the npm init command. Required packages such as Express.js, Mongoose, CORS, JWT, and other dependencies are installed to support backend functionality.

Express Server Creation

An Express.js server is configured to handle client requests and provide API services. Middleware such as JSON parsing and CORS is added to process request data and enable communication between the frontend and backend.

Route Development

Separate route files are created for different modules, including authentication, complaints, users, agents, and chat functionality. These

routes define the available API endpoints and direct requests to the appropriate controllers.

Controller Implementation

Controllers are developed to handle application logic. They receive requests from routes, process the required operations, interact with database models, and send responses back to the client.

Database Model Creation

Mongoose schemas and models are created for entities such as Users, Complaints, Agents, Messages, and Admins. These models define the database structure and facilitate interaction with MongoDB.

Complaint Management

The complaint module allows users to create complaints, view complaint details, track status updates, and communicate with assigned agents. Agents can update complaint progress and provide resolutions.

User Authentication

Authentication is implemented using JSON Web Tokens (JWT). Users and agents can register and log in securely, while protected routes are accessible only to authorized users.

Admin Operations

Admin-specific features are developed to manage users, agents, and complaints. Administrators can assign complaints, monitor platform activities, and oversee complaint resolution processes.

Chat Functionality

A messaging system is implemented to enable communication between users and agents. Messages are stored in the database and linked to the relevant complaint for easy tracking.

Error Handling

Error-handling middleware is added to catch application errors and return meaningful error messages with appropriate HTTP status codes, improving system reliability and debugging.

System Testing

All modules are tested to ensure smooth functionality. APIs are verified, database operations are validated, and authentication and complaint workflows are checked for accuracy and performance.

Configure MongoDB

Install Mongoose

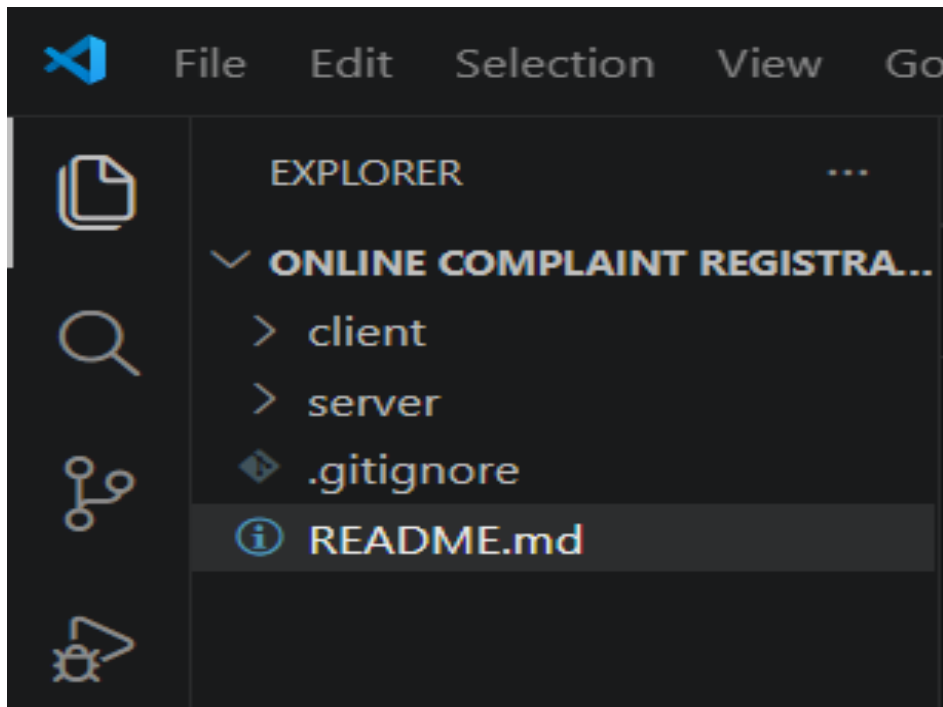
Mongoose is used to connect the Node.js application with the MongoDB database and perform database operations.

Run the following command in the terminal:

- `npm install mongoose`

After installation, Mongoose can be used to create database schemas, models, and connect the application to MongoDB.

CREATING FOLDER STRUCTURE

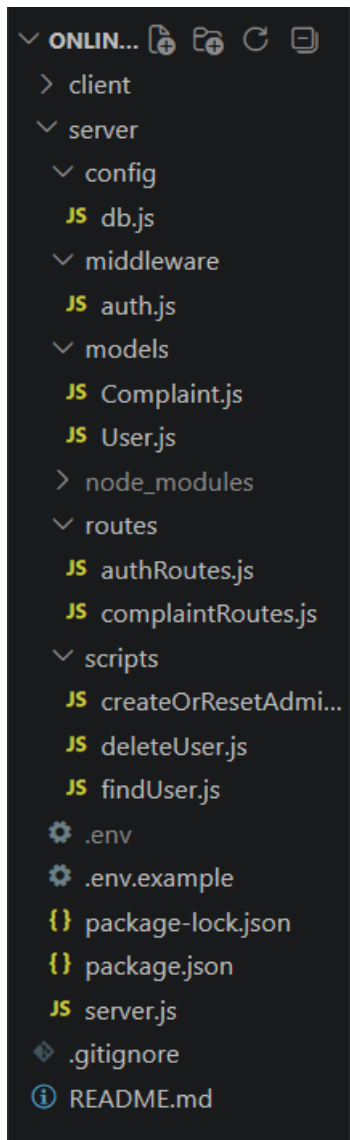


This is the **Online Complaint Registration** project. The project is organized into different folders and files to separate the frontend, backend, and configuration files, making the application easy to develop, manage, and maintain.

- **client/** – Contains the frontend of the application, developed using **React**. It includes the user interface, pages, reusable components, routing, and styling files that provide an interactive experience for users.
- **server/** – Contains the backend of the application, built with **Node.js** and **Express.js**. It handles API requests, database connectivity, user authentication, complaint management, and other server-side operations.
- **.gitignore** – Specifies the files and folders that Git should ignore, such as **node_modules**, **.env**, and other temporary files. This prevents unnecessary or sensitive files from being uploaded to the Git repository.
- **README.md** – Contains the project documentation, including the project overview, features, technologies used, installation steps, setup instructions, and guidelines for running the application.

This project structure follows a standard full-stack development approach by separating the frontend and backend into independent modules. It improves code organization, enhances readability, simplifies maintenance, supports collaboration among developers, and makes the application easier to test, update, and deploy efficiently.

SERVER FOLDER STRUCTURE



The figure shows the **server folder** of the **Online Complaint Management System**. This folder contains the backend source code developed using **Node.js**, **Express.js**, and **MongoDB**. It is organized into different folders and files, where each component performs a specific task in the application.

config/ – Contains the database configuration file (**db.js**) used to connect the application with the MongoDB database.

middleware/ – Contains middleware functions such as **auth.js**, which is used for user authentication and authorization before processing requests.

models/ – Defines the MongoDB schemas for the application's collections, including:

- **User.js** – Stores user information such as name, email, password, and role.
- **Complaint.js** – Stores complaint details such as title, description, category, and status.

routes/ – Contains API route definitions that handle user and complaint-related requests.

- **authRoutes.js** – Handles user registration and login operations.
- **complaintRoutes.js** – Handles complaint creation, viewing, updating, and deletion.

scripts/ – Contains utility scripts used for administration and database management.

- **createOrResetAdmin.js** – Creates or resets the admin account.
- **deleteUser.js** – Deletes a user from the database.
- **findUser.js** – Searches for a user in the database.

node_modules/ – Stores all npm packages and dependencies required by the backend application.

.env – Stores environment variables such as the MongoDB connection URL, JWT secret key, and server port.

.env.example – Provides a sample environment configuration file for developers.

.gitignore – Specifies files and folders that should not be tracked by Git, such as **node_modules** and **.env**.

package.json – Contains project information, dependencies, and scripts required to run the backend server.

package-lock.json – Maintains the exact versions of installed npm packages to ensure consistency across different systems.

server.js – The main entry point of the backend application. It initializes the Express server, connects to the MongoDB database, configures middleware, registers API routes, and starts the server.

README.md – Contains project documentation, setup instructions, and usage details.

The server folder follows a **modular architecture**, where each component has a specific responsibility. This structure improves code organization, simplifies maintenance and debugging, promotes code reusability, and makes the application easier to scale and enhance in the future.

CREATE DATABASE CONNECTION

```
const mongoose = require("mongoose");

const connectDB = async () => {
  try {
    const uri = process.env.MONGO_URI || "mongodb://localhost:27017/complaint_management";
    await mongoose.connect(uri);
    console.log("MongoDB connected successfully");
  } catch (error) {
    console.error("MongoDB connection failed:", error.message);
    process.exit(1);
  }
};

module.exports = connectDB;
```

Database Connection (db.js)

The figure shows the **db.js** file of the **Online Complaint Management System**. This file is responsible for establishing a connection between the application and the **MongoDB** database using **Mongoose**.

- Imports **Mongoose** to interact with the MongoDB database.
- Creates the **connectDB()** function to establish the database connection.
- Uses **mongoose.connect()** to connect to MongoDB using the connection URL stored in environment variables.
- Displays a success message when the database connection is established successfully.

- Uses a **try-catch block** to handle connection errors and display appropriate error messages.
- Exports the **connectDB()** function so it can be imported and used in the main server file (**server.js**).

This file plays an important role in the application because it provides a secure and reliable connection between the backend server and the MongoDB database. It enables the system to store, retrieve, update, and manage user and complaint data efficiently.

Schema and models

```
const mongoose = require("mongoose");
const bcrypt = require("bcryptjs");

const userSchema = new mongoose.Schema(
  {
    name: { type: String, required: true, trim: true },
    email: { type: String, required: true, unique: true, lowercase: true, trim: true },
    password: { type: String, required: true },
    phone: { type: String, trim: true },
    role: { type: String, enum: ["user", "agent", "admin"], default: "user" },
  },
  { timestamps: true }
);

userSchema.pre("save", async function (next) {
  if (!this.isModified("password")) return next();
  const salt = await bcrypt.genSalt(10);
  this.password = await bcrypt.hash(this.password, salt);
  next();
});

userSchema.methods.comparePassword = function (candidatePassword) {
  return bcrypt.compare(candidatePassword, this.password);
};

module.exports = mongoose.model("User", userSchema);
```

```

const mongoose = require("mongoose");

const complaintSchema = new mongoose.Schema(
  {
    title: { type: String, required: true, trim: true },
    description: { type: String, required: true },
    category: {
      type: String,
      enum: ["Billing", "Technical", "Service", "Product", "Other"],
      default: "Other",
    },
    priority: {
      type: String,
      enum: ["Low", "Medium", "High", "Urgent"],
      default: "Medium",
    },
    status: {
      type: String,
      enum: ["Open", "In Progress", "Resolved", "Closed", "Rejected"],
      default: "Open",
    },
    complainant: { type: mongoose.Schema.Types.ObjectId, ref: "User", required: true },
    assignedAgent: { type: mongoose.Schema.Types.ObjectId, ref: "User", default: null },
    attachments: [{ type: String }],
    comments: [
      {
        author: { type: mongoose.Schema.Types.ObjectId, ref: "User" },
        message: String,
        createdAt: { type: Date, default: Date.now },
      },
    ],
    resolutionNote: { type: String, default: "" },
  },
  { timestamps: true }
);

module.exports = mongoose.model("Complaint", complaintSchema);

```

User Model (User.js)

The **User Model** is used to store user information in MongoDB.

It contains:

- **Name** – Stores the user's full name.
- **Email** – Stores a unique email address and validates the email format.
- **Password** – Stores the user's password securely after hashing using bcrypt.

- **Role** – Defines the user type, such as **User** or **Admin**.
- **Timestamps** – Automatically stores account creation and update times.

It also includes:

- **Password Hashing** – Encrypts the password before saving it to the database.
- **Password Comparison** – Compares the entered password with the stored encrypted password during login.

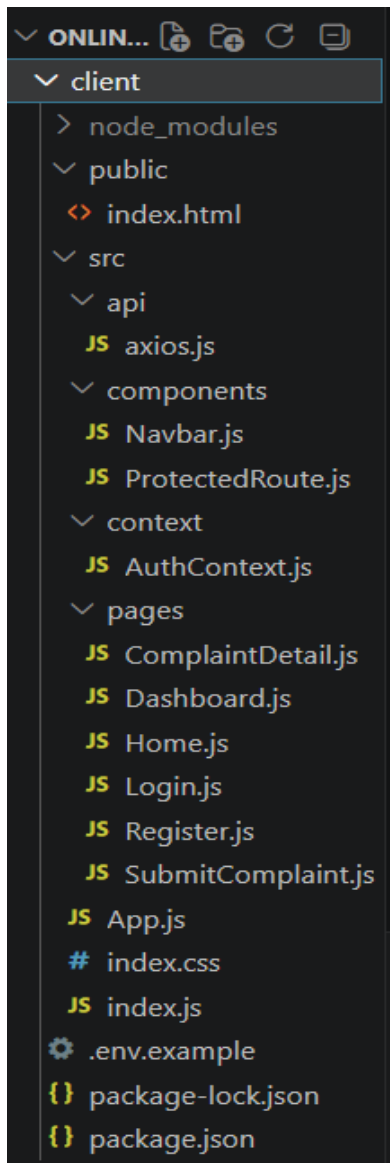
Complaint Model (Complaint.js)

The **Complaint Model** is used to store complaint details in MongoDB.

It contains:

- **User ID** – Links the complaint to the user who submitted it.
- **Reference Number** – A unique identifier for each complaint.
- **Title** – The subject or title of the complaint.
- **Category** – The complaint category, such as Technical, Billing, or Service.
- **Description** – Detailed information about the complaint.
- **Attachment URL** – Stores the path or URL of uploaded files or documents.
- **Status** – Tracks the complaint status, such as Pending, In Progress, or Resolved.
- **Timestamps** – Automatically stores complaint creation and update times.

CLIENT FOLDER STRUCTURE



The figure shows the frontend folder structure of the **Online Complaint Registration System** developed using **React.js**.

Client Folder

The **client** folder contains all frontend source code and resources required for the user interface of the application.

Public Folder

The **public** folder contains static files that are directly accessible by the browser.

Index.html

The **index.html** file is the main HTML file that loads the React application into the browser.

API Folder

The **api** folder contains the **axios.js** file, which is used to configure Axios and handle communication between the frontend and backend server.

Components Folder

The **components** folder contains reusable UI components used throughout the application.

- **Navbar.js** – Provides navigation links for different pages.
- **ProtectedRoute.js** – Restricts access to protected pages and allows only authenticated users.

Context Folder

The **context** folder contains **AuthContext.js**, which manages user authentication, login status, and user information across the application.

Pages Folder

The **pages** folder contains all application pages.

- **Home.js** – Displays the homepage.
- **Register.js** – Allows users to create a new account.
- **Login.js** – Allows users to log in to the system.
- **Dashboard.js** – Displays user complaints and complaint status.
- **SubmitComplaint.js** – Provides a form to submit new complaints.
- **ComplaintDetail.js** – Displays detailed information about a selected complaint.

Main Files

- **App.js** – Main application component responsible for routing and rendering pages.
- **index.js** – Entry point of the React application.
- **index.css** – Contains global styling for the application.

Configuration Files

- **.env.example** – Contains sample environment variables.

- **package.json** – Stores project metadata, dependencies, and scripts.
- **package-lock.json** – Maintains exact versions of installed packages.

PROJECT EXECUTION

Steps for Project Execution:

Step 1 : Download the Project

```
cd App
```

Step 2 : Install Client Dependencies

```
cd client
```

```
npm install
```

```
npm start
```

Step 3 : Install Server Dependencies

```
cd server
```

```
npm install
```

```
npm start
```

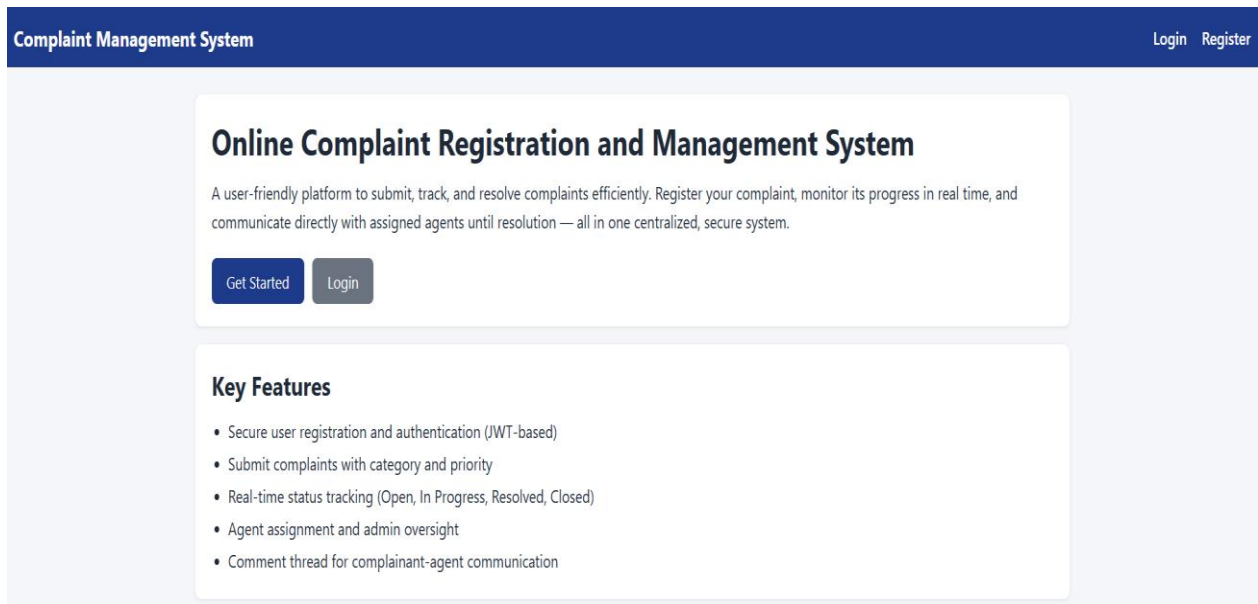
Step 4 : Create a .env file with your MongoDB URI:

```
MONGO_URI=mongodb://localhost:27017/OnlineComplaintRegistration
```

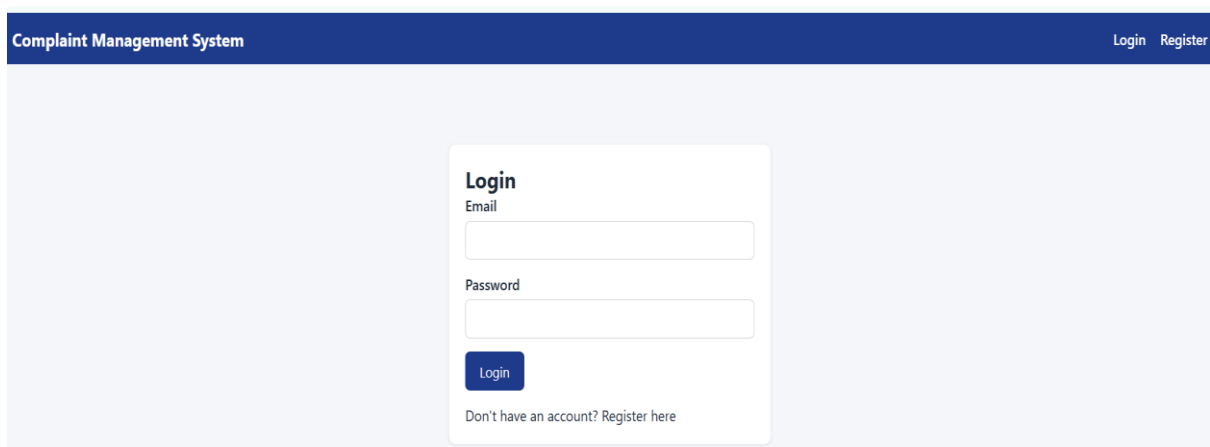
DEMO SCREENSHOTS

Output ScreenShots:

- **Landing Page** : Displays a welcome message with options to login or register. Serves as the entry point for all users (User, Admin).



- **Login Page** : Shows a form with email and password fields. On successful login, redirects user to their respective dashboard.



- **Registration Page** : Provides a form for new users to sign up with basic details. Shows a success message and redirects to the login page upon registration.

Complaint Management System

LoginRegister

Create an Account

Full Name

Email

Phone

Password

Register

Already have an account? Login here

- **Common Dashboard For Complaint** : Users can submit new complaints and view their complaint history. Each complaint displays its status (Pending, Assigned, Resolved) and assigned agent.

Complaint Management System

DashboardSubmit Complaint

Submit a New Complaint

Title

Description

Category

Other

Priority

Medium

Submit Complaint

My Complaints

Title	Category	Priority	Status	Created	
water leakage	Service	Medium	Open	7/14/2026	View

All Statuses

All Statuses

Open

In Progress

Resolved

Closed

Rejected

•**Admin Dashboard** : Shows complaints assigned to the agent with their details. Admin can update the status of complaints (e.g., In Progress, Resolved).

All Complaints

All Statuses ▾

Title	Category	Priority	Status	Complainant	Created	
Internet Service Complaint	Technical	High	Open	Kowshisk	7/14/2026	<button>View</button>
Electricity Supply Issue	Technical	Urgent	In Progress	Harshitha	7/14/2026	<button>View</button>
water leakage	Service	Medium	Resolved	janu	7/14/2026	<button>View</button>

Internet Service Complaint

Open

Category: Technical | Priority: High

Internet connectivity has been very slow and frequently disconnects. Please check the network and restore proper service.

Submitted by Kowshisk on 7/14/2026, 7:12:56 PM

Update Status:

Open ▾

Delete Complaint

Comments / Updates

No comments yet.

Add a comment...

Post Comment

DEMO LINKS

<https://drive.google.com/file/d/1djkdwPBcNnrOEyyfncknYvtNZvbit8Qw/view?usp=sharing>

